Activity 4

Systemintegration, AAA & SSO

Network Technology

2026

1. Introduction

This project is about setting up and testing a system for authentication, authorization and accounting, i.e. AAA, as well as Single Sign-On, abbreviated SSO. The purpose is to understand how these technologies work in practice and how they can be integrated in a network environment.

The environment is simulated with three virtual machines running in Proxmox. Two servers run Ubuntu Server 22.04 and one runs Ubuntu Desktop 22.04. One server is used for FreeRADIUS, an AAA system, and the other for Keycloak, an identity management system that also supports the RADIUS protocol. Ubuntu Desktop is used as a client machine to test the login flow.

The project is divided into four parts: implementation of AAA with FreeRADIUS, configuration of SSO with Keycloak and two web applications, an analysis of how system integration affects security and deployment, and a reflection on incident management.

---

2. AAA Solution with FreeRADIUS

2.1 What is AAA?

AAA is a framework for network access control consisting of three parts. Authentication refers to verifying who a user is. Authorization determines what the user is allowed to do. Accounting logs what the user has done and when.

FreeRADIUS is an open-source implementation of the RADIUS protocol (RFC 2865) and is widely used in enterprise networks to manage access to routers, switches and VPN solutions.

2.2 Server Configuration

FreeRADIUS was installed on VM1 with IP address 192.168.1.111. After installation, three users were configured with different permission levels in the file /etc/freeradius/3.0/users. Passwords were stored as MD5 hashes to avoid saving them in plaintext.

User	Role	Permission
admin	Network Administrator	Full access
user1	Standard user	Limited access
guest	Guest	Minimal access

An important security setting was to change the shared secret from the default value testing123 to a stronger password. In addition, protection against the BlastRADIUS attack (CVE-2024-3596) was enabled by setting require_message_authenticator = true in clients.conf.

2.3 Testing of User Roles

Authentication was tested using the radtest tool, which sends RADIUS requests directly from the command line. The tests showed that all three users were authenticated correctly, and that incorrect passwords resulted in Access-Reject as expected.

2.4 Accounting

The accounting component of FreeRADIUS logs session information such as start and stop times for connections. In the test environment, logging was enabled and verified via /var/log/freeradius/radius.log. A limitation in the lab is that full accounting requires a real NAS device, such as a router or switch, that sends Accounting-Request packets to the RADIUS server. In our simulated environment, only the authentication part was tested.

---

3. SSO Configuration with Keycloak

3.1 What is Keycloak and SSO?

Keycloak is an open-source Identity Provider (IdP) developed by Red Hat. It supports the OAuth 2.0, OpenID Connect (OIDC) and SAML 2.0 protocols. Single Sign-On means that a user logs in once and then gains access to multiple systems without having to authenticate again.

In this project, Keycloak acts as a central identity service. When a user logs in via Keycloak, a JWT token is issued that the connected applications accept. This means that the user only needs to enter their credentials once, regardless of how many applications are connected.

3.2 Installation and Configuration

Keycloak version 26.4.0 was installed on VM2 with IP address 192.168.1.112. Since the server runs in a virtual environment without a public domain, Keycloak was configured in development mode with HTTP and a built-in H2 database. It is important to note that this is not a production configuration; in a live environment, HTTPS with a valid certificate and an external database such as PostgreSQL would be required.

A realm called lab-realm was created to isolate the environment from the built-in master realm, which is used for administration. In lab-realm, two clients were created, app-one and app-two, representing the two web applications. Three users were created with associated roles: admin-role, user-role and guest-role.

3.3 RADIUS Integration in Keycloak

An important part of the project was integrating the RADIUS protocol directly into Keycloak via a plugin called keycloak-radius-plugin. This allows Keycloak to function as a RADIUS server in parallel with its role as an OIDC provider. It means that Keycloak can handle both network authentication via RADIUS and application login via SSO with one and the same system and user database.

To enable RADIUS, a client of type RADIUS Protocol was created in Keycloak, and the configuration file radius.config was updated with the correct shared secret and ports. The server now listens on port 1812 for authentication and 1813 for accounting.

3.4 The Two Web Applications

To demonstrate SSO, two web applications were created in Python using Flask and authlib. The applications are called Discover Sicily, a tourist portal system, and La Tavola Siciliana, a restaurant booking service. Both applications delegate authentication to Keycloak via OIDC.

The flow works as follows: the user clicks Sign In in one of the applications, is redirected to the Keycloak login page, enters their credentials, and is redirected back to the application

with a JWT token. If the user then opens the other application in the same browser session, Keycloak recognises the session and logs in automatically without asking for credentials again. This is the core of SSO.

3.5 Login Flow

The complete login flow is described below, step by step:

- Usern öppnar app-one på http://192.168.1.113:8081 och klickar på Sign In.
- App-one redirects the browser to Keycloak at http://192.168.1.112:8080/realms/lab-realm/protocol/openid-connect/auth.
- Keycloak displays its login page where the user enters their username and password.
- Keycloak verifies the credentials against its user database and issues a JWT token.
- The browser is redirected back to app-one via the callback URL with an authorisation code.
- App-one exchanges the code for an access_token and an id_token via Keycloak's token endpoint.
- Usern är nu inloggad i app-one och Keycloak har skapat en session.
- Usern öppnar app-two på http://192.168.1.113:8082 och klickar på Sign In.
- App-two redirects to the same Keycloak endpoint, but Keycloak recognises the session and skips the login step.
- Usern är inloggad i app-two utan att ange uppgifterna igen. SSO är verifierat.

---

4. System Integration Analysis

4.1 Security

Centralising authentication in a system like Keycloak has both advantages and disadvantages from a security perspective. The main advantage is that all security updates and policies are managed in one place. If a password needs to be changed or an account blocked, it is done in Keycloak and takes effect immediately for all connected systems.

The disadvantage is that Keycloak becomes a single point of failure, i.e. a critical component. If the system is compromised or goes down, all connected services are affected. This places high demands on the Keycloak server being well-secured, up to date and redundant in a production environment.

During the course of the project, a couple of security weaknesses were identified and addressed. In FreeRADIUS, the default secret testing123 was replaced with a stronger password, and protection against the BlastRADIUS attack was enabled. This is a good example of how default configurations are rarely sufficient and should always be reviewed.

Passwords in FreeRADIUS were stored as MD5 hashes. This is better than plaintext, but MD5 is now considered weak and should in a live environment be replaced with bcrypt or an integration with an LDAP directory using more modern hashing.

In Keycloak, everything ran via HTTP without TLS. This is acceptable in an isolated lab network but completely unacceptable in a production environment where passwords and tokens would otherwise be transmitted unencrypted.

4.2 Usability

SSO significantly improves the user experience. In this project, it was demonstrated that a user who logs in to one application is automatically logged in to the other. In an organisation

with ten or twenty internal systems, this is an enormous improvement compared to having separate logins for each system.

Centralisation also simplifies administration. Creating, modifying or deleting a user is done in one place, rather than having to do it in each system separately. This reduces the risk of forgetting to remove access when an employee leaves.

A potential disadvantage of SSO is that if the user logs out, it must happen across all systems, otherwise a session may remain active. This requires that logout is handled centrally, which authlib and Keycloak support but which must be configured correctly.

4.3 Troubleshooting

Centralised authentication facilitates troubleshooting because all logging is gathered in one place. In Keycloak, there is an Events section in the administration portal that logs all login attempts, both successful and failed, with a timestamp and IP address. This makes it easy to trace failed login attempts or identify where in the flow a problem occurs.

In FreeRADIUS, events are logged in /var/log/freeradius/radius.log. During the project, the logs were invaluable for troubleshooting configuration problems, including when the BlastRADIUS warnings appeared and when modules were not loading correctly.

A challenge with a centralised system is that troubleshooting requires knowledge of the entire chain. If a user cannot log in, the problem may lie in the application, in Keycloak, in the RADIUS client or in the network. This requires that operations staff have an understanding of all parts.

---

5. Incident Management and Guidance

5.1 Incident Response Procedures

In a system with centralised authentication, it is important to have clear procedures for what happens when an incident occurs. The most important scenarios and how they are handled are described below.

Compromised Account

If a user account is suspected of being compromised, for example due to a data breach or an unusual login pattern, the account should be immediately disabled in Keycloak. Disabling it in one place is sufficient to cut off access to all connected systems. Active sessions should also be manually terminated via the Sessions tab in Keycloak.

In parallel, the event log in Keycloak should be reviewed to determine which systems were accessed and during what time period. This helps assess the extent of the damage.

Service Outage

If Keycloak or FreeRADIUS stops working, all dependent systems are affected. The system status can be checked with sudo systemctl status keycloak and sudo systemctl status freeradius respectively. For a crashed system, the logs can be inspected with sudo journalctl -u keycloak to identify the cause of the failure.

In a production environment, there should be a fallback mechanism, such as a local administrator login on each system, to enable access even when the central server is down.

Failed Login Attempts

Repeated failed login attempts may indicate a brute-force attack. Keycloak has built-in support for locking accounts after a number of failed attempts via Brute Force Protection under Realm Settings. This should always be enabled in a live environment.

5.2 Guidance for Users

An important part of operations is communicating with users in a clear way. Below are some recommendations that are relevant in connection with this system.

- Inform users that they now have a single login for all systems and that they should never share their credentials with others.
- Explain that if they log in on a computer shared with others, they should always log out properly, as the SSO session would otherwise grant access to all connected systems.
- Encourage users to report suspicious activity, such as unexpected login notifications, directly to the operations staff.
- When changing a password, it is sufficient to change it in one system, i.e. Keycloak, for it to apply everywhere.
- If a user is locked out of their account, they should contact the administrator rather than attempting to reset the password themselves, as the account may be locked due to a security measure.

---

6. Conclusion

The project has demonstrated how AAA and SSO can be implemented in a simulated environment with FreeRADIUS and Keycloak. The system integration means that one and the same identity service handles authentication for both network devices via RADIUS and web applications via OIDC.

The greatest advantages of a centralised system are improved user experience, simpler administration and better logging. The most important risks are that the system becomes a critical component and that a compromise has consequences for all connected services.

During the course of the project, we encountered a number of technical challenges, including configuration issues in FreeRADIUS and compatibility questions with the Keycloak plugin. This provided a practical understanding of how complex it can be to set up this type of infrastructure in reality, and how important careful documentation and logging are.